Recognizing Landmarks with a Convolutional Neural Network

Carlos Manuel Martínez Vázquez
Instituto Tecnológico y de Estudios Superiores de Monterrey

November 5, 2026

Abstract

This work presents the implementation for famous landmark recognition from a photograph. The first iteration works upon a convolutional neural network (CNN). The data comes from the Google Landmarks Dataset v2 (GLDv2), the dataset that is behind Kaggle's *Landmark Recognition* competitions, from which the 50 landmarks with the most photographs were selected for this project: 7,500 images in total, 150 per landmark, later raised to 200 per landmark (10,000 images). A small CNN is first trained from scratch with Keras and TensorFlow, evaluated on held-out validation and test sets, and diagnosed for bias and variance from its training and validation curves. Regularization is then applied to correct the diagnosis. The baseline reaches 46.3% test accuracy; a deeper network with data augmentation and dropout, trained on 200 images per landmark, reaches 61.9% with a train–validation gap of 11 points instead of 23.

1 Introduction

Landmark recognition is the task of identifying which place, building, monument or natural site in specific appears within a photograph. It underlies photo organization and geographic tagging in services such as Google Photos, it is harder than just general object classification such as just two photographs of the same landmark such as a cathedral may share almost nothing at the actual pixel level (different angle, lighting, crowds, season, etc.) while two different cathedrals can look very similar to each other.

This work then approaches this problem as a multiclass image classification task: given a photograph of one of the 50 selected landmarks, predict which one it is. The first model is a convolutional neural network, just the standard deep learning architecture for images, implemented with Keras on TensorFlow.

The full GLDv2 dataset is far too large for a single laptop, so the first part of the work concerns building a subset that is small enough to train locally and still be balanced and representative of the full set. The second part trains the baseline CNN, diagnoses it, and improves it.

2 ETL

2.1 Data source

GLDv2 [1] was published by Google in 2020. Its photographs come from Wikimedia Commons, and each one is labeled with the Wikimedia category of the landmark it shows. The dataset is distributed as images plus metadata files:

- `train.csv`: 4,132,914 images of 203,094 landmarks, with the URL of every image.
- `train_clean.csv`: a cleaned subset of 1,580,470 images of 81,313 landmarks, with noisy labels removed.
- `train_label_to_category.csv`: the Wikimedia category name of each landmark.

The images alone weigh roughly 500 GB. Only the metadata (about 570 MB) is actually downloaded in full; images then are fetched individually for the selected landmarks to build the subset of 50 landmarks.

2.2 Landmark selection

The cleaned set was used (`train_clean.csv`), since in the raw one several of the largest "landmarks" are generic categories containing some unrelated photographs. Even so, the distribution of images per landmark is extremely long-tailed: the median landmark has only 9 photographs, far too few to train a CNN on to classify correctly. The landmarks were therefore ranked by number of images and the top 50 kept for our subset. Three entries of that ranking are not places but photo archives or organizations and were excluded: *Media contributed by the ETH-Bibliothek* (the largest category, with 6,272 images), *Episcopal Diocese*

of Southwest Florida and *SBB Historic*. The 50 selected landmarks have between 492 and 2,231 images each (Table 1).

Landmark	Images
Corktown, Toronto	2,231
Noraduz Cemetery	1,758
Museum of Folk Architecture, Pyrohiv	1,741
Salève	1,135
Lviv	1,088
...	
Niagara Falls	944
Grand Canyon	888
Golden Gate Bridge	888
Edinburgh Castle	734
Eiffel Tower	731
...	
Saiful Muluk Lake	492

Table 1: Some of the 50 selected landmarks and their number of images in `train_clean.csv`. The full list is in `data/classes.csv`.

The subset selection mixes very different kinds of places: monuments (Eiffel Tower), natural sites (Grand Canyon, Niagara Falls), religious buildings, fortresses, and a few broad areas such as a city (Lviv) or a neighborhood (Corktown). The last group is expected to be the hardest for the model, since its photographs do not share specific characteristics such as a particular building.

2.3 Sampling and download

From each landmark 150 images were sampled at random with a fixed seed, so that every class has the same weight and accuracy is not dominated by the largest ones. Images then were downloaded from Wikimedia Commons as 330 pixel wide thumbnails, which are 5 to 60 times lighter than the originals. Most are close to the 224×224 we have as input of the network, but 38% have a short side below 224 pixels and are slightly enlarged when resized. About 98% of the GLDv2 URLs (the dataset isn't explicitly pure photos, it is a defined list of links) were still valid; 30% more images than needed were requested just to cover the dead ones. Wikimedia limits how fast images can be requested, so about 470 of them were taken instead from the copy of GLDv2 hosted on Kaggle, which actually contains the same files. The download continued while the first models trained, and once every landmark had at least 200 images the subset was raised to 200 per landmark for this iteration.

2.4 Train, validation and test sets

GLDv2's own test set was not used: most of its images are deliberately not landmarks, and very few belong to the 50 selected classes. The images were instead split 70/15/15 and this split was *stratified*: each landmark was divided on its own, so all three sets contain every landmark in the same proportion (Table 2).

	150 per landmark		200 per landmark	
Set	Images	Each	Images	Each
Training	5,250	105	7,000	140
Validation	1,100	22	1,500	30
Test	1,150	23	1,500	30

Table 2: Stratified 70/15/15 split of both subsets.

The 200 subset is split again from scratch, so its validation and test sets are not the same as those of the 150 subset; results of the two are close but not strictly comparable to each other.

The training set is used to fit the weights, the validation set to monitor training and choose between models, and the test set only to report the final results. A known risk of GLDv2 is that a landmark may contain near-duplicate photographs (the same scene uploaded twice, or burst shots), which could end up on both sides of the split and then inflate the test score. Also to note, future iterations will include more images per landmark.

2.5 Preprocessing

Each image is center-cropped to a square, so that it is not stretched, and resized to 224×224 pixels with three color channels. Pixel values are scaled from $[0, 255]$ to $[0, 1]$. The crop and resize are done when the image is already loaded, by equivalent Keras options in training and in the predict file (`predict.py`), and the scaling is a layer of the model itself, so a photograph given to the trained model at prediction time goes through the same transformation as the training images.

3 Convolutional Neural Network

3.1 Why a CNN

A fully connected network treats an image as a flat list of 150,528 numbers: it has no notion that neighboring pixels are related, and a single layer of just purely 256 neurons would already need 38 million weights. A convolution layer instead slides small filters fully across the image, and the same filter is reused at every position. This has two consequences: the number of weights no longer depends on the image size, and a pattern learned in one place (an arch, a window) is recognized anywhere in the photograph.

3.2 Building blocks

Convolution. A filter K of size 3×3 over C input channels produces one output map:

$$y_{i,j} = \sum_{c=1}^{C} \sum_{u=-1}^{1} \sum_{v=-1}^{1} K_{u,v,c}\, x_{i+u,\, j+v,\, c} \tag{1}$$

A layer with F filters produces F such maps and has $3 \cdot 3 \cdot C \cdot F$ weights.

Batch normalization. Each map is standardized over the batch and then rescaled with two learned parameters γ, β, which keeps the inputs of the next layer on a stable scale and speeds up training [3]:

$$\hat{x} = \gamma \, \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta \tag{2}$$

ReLU. The activation $\max(0, x)$ makes the network non-linear.

Max pooling. Each 2×2 window is replaced by its maximum, halving the width and height. After every block a filter therefore covers a region of the original photograph twice as large.

Global average pooling. The last block outputs 14×14 maps; global average pooling [4] reduces each to its mean, giving one number per filter. The usual alternative, flattening the maps into a dense layer, would add about 2.5 million weights, more than six times the rest of the network.

Softmax. The output layer has one neuron per landmark, and softmax turns its outputs z_k into probabilities that sum to one:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{50} e^{z_j}} \tag{3}$$

3.3 Architecture

The baseline network has four convolutional blocks (Table 3). Every block halves the image and doubles the number of filters, from 32 to 256: the first layers respond to edges and colors, the later ones to larger shapes such as arches, towers and rooflines.

The *deep* version used in Section 4 keeps the same four blocks but stacks two convolutions in each one before pooling, in the style of VGG [7]: 8 convolutional layers and 1,187,986 parameters. Two 3×3 convolutions see a 5×5 region of their input with fewer weights than a single 5×5 filter, and add one more non-linearity.

3.4 Training

The network is trained by minimizing the categorical cross-entropy, the multi-class version of the cost used in logistic regression:

$$J = -\frac{1}{m} \sum_{i=1}^{m} \log p_{i,\, y_i} \tag{4}$$

Layer	Output	Parameters
Input, rescaling	$224 \times 224 \times 3$	0
Block 1: conv 32	$112 \times 112 \times 32$	992
Block 2: conv 64	$56 \times 56 \times 64$	18,688
Block 3: conv 128	$28 \times 28 \times 128$	74,240
Block 4: conv 256	$14 \times 14 \times 256$	295,936
Global average pooling	256	0
Dense, softmax	50	12,850
Total		**402,706**

Table 3: Baseline CNN. Each block is a 3×3 convolution, batch normalization, ReLU and 2×2 max pooling; its parameters include the batch normalization ones.

where p_{i,y_i} is the probability the model assigns to the true landmark of image i. A model that guesses uniformly among the 50 classes has a cost of $\log 50 = 3.91$, which is the starting point of training.

The optimizer is Adam [6] with batches of 32 images. Training runs for at most 50 epochs and stops early when the validation cost has not improved for a number of consecutive epochs (the *patience*); the weights of the best epoch are then restored. The baseline used a learning rate of 10^{-3} and a patience of 5. Its validation curve jumped from epoch to epoch: batch normalization keeps a running average of μ_B and σ_B^2 to use at prediction time, and with Keras' default momentum of 0.99 that average lags behind weights that change quickly. The later models use a learning rate of 3×10^{-4} and a momentum of 0.9, and a patience of 10, since augmented models improve more slowly and noisily (Section 4.3).

Four metrics are reported. *Accuracy* is the fraction of images whose most probable landmark is the correct one, and *top-5 accuracy* the fraction whose correct landmark is among the five most probable; random guessing scores 2% and 10% respectively. Accuracy alone does not show whether the model is right on every landmark or just on the easy ones, so the *macro-F1* is also reported: the F1 score of each landmark, averaged with equal weight. Finally, the Kaggle competitions score with the *Global Average Precision* (GAP) [9]: every image gets one prediction and a confidence, the predictions are sorted from most to least confident, and

$$\mathrm{GAP} = \frac{1}{M} \sum_{i=1}^{N} P(i)\, \mathrm{rel}(i) \tag{5}$$

where $P(i)$ is the precision among the first i predictions, $\mathrm{rel}(i)$ is 1 if prediction i is correct and 0 if not, and M the number of images. GAP rewards a model that is confident when it is right and unsure when it is wrong. It was implemented by hand and checked against a small example computed on paper.

3.5 Framework

The model is implemented in Keras 3 [10, 2] on TensorFlow 2.18, and trained on the GPU of an Apple M2 Pro through the `tensorflow-metal` plugin. Images are loaded with `keras.utils.image_dataset_from_directory`, which reads one folder per landmark and uses the folder name as the label.

4 Model Evaluation

4.1 Results

Every model trained is listed in Table 4, in the order they were trained. Training accuracy is measured after training, on the training images without augmentation or dropout, so it is comparable with validation and test. The first row was a trial run while the images were still downloading, with only 73 training images per landmark.

4.2 Bias and variance

As in logistic regression, both can be read from the training and validation curves: the gap between them measures variance, and the level at which they settle measures bias.

The baseline has high bias. The expectation was overfitting, since the network has 400 thousand parameters and only 105 images per landmark. That is not what happened: the gap is only 5.8 points, but the network cannot even fit its own training images, which stay at 53.6%. Its validation cost stopped falling at 1.99, about half of the 3.91 of random guessing. The productive direction was therefore more capacity, not regularization, which is where the deep version of the network comes from.

The deep network has high variance. Doubling the convolutions per block raised test accuracy by 23 points, to 69.3%, but the training accuracy went up to 94.1% while validation stayed at 68.7%: a gap of 25 points. The network now memorizes the training images, and the problem turned into the opposite one, which is the one regularization is meant for.

4.3 Improving the model: regularization

Each technique below targets variance; they were added one at a time so the effect of each could be measured on the validation set.

Data augmentation. Every time a training image is used, it is randomly flipped horizontally, rotated by up to $\pm 18°$, zoomed by up to 20%, shifted by up to 10% and has its contrast changed by up to 20%. These are differences that two tourists' photographs of the same landmark naturally have, so the model is shown many plausible versions of each photograph instead of memorizing the exact pixels of 105 of them. The transformations are layers of the model that are only active during training.

Dropout. During training, a random 50% of the 256 pooled features is set to zero at every step [5], so the output layer cannot rely on any single feature and has to spread the evidence across many.

L2 regularization. The penalty $\frac{\lambda}{2}\sum w^2$ can also be added to the cost, as in the previous logistic regression project. It is implemented but was not tested in this iteration.

The first attempt, augmentation on the baseline, stopped at epoch 9 with 25.7% test accuracy. An augmented model sees a different version of every image each epoch, so its validation cost is noisier, and 5 epochs without improvement happened by chance long before it had converged. This, together with the jumping validation curve of the baseline, led to the patience of 10 and the lower learning rate and momentum of Section 3.4.

On the deep network both techniques did what was expected with the gap: augmentation reduced it from 25.4 to 18.6 points and dropout from 18.6 to 10.9. They also lowered validation accuracy, from 68.7% to 63.2% and 57.8%: with 105 images per landmark, regularization removed variance but added bias. Raising the subset to 140 training images per landmark improved both versions, and the regularized one gained more on test (+5.8 points, against +3.6 for the unregularized one).

The model kept is the deep network with augmentation and dropout trained on 140 images per landmark (Figure 1). Its test accuracy, 61.9%, is lower than the 72.9% of the same network without regularization, but its gap is half as large, 11.5 points against 23.3. The network without regularization has already memorized its training set (95.8%), so it has little room left to improve. The curves of the regularized one stay together for the first 20 epochs, and the run ended at the limit of 50 epochs with its training cost still falling, although its validation cost had not improved since epoch 44.

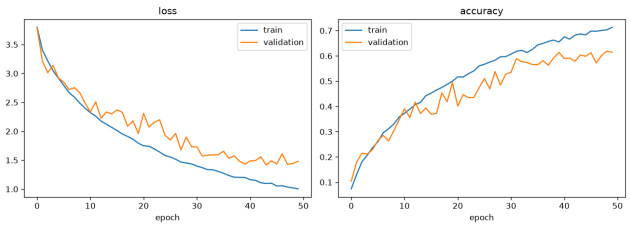


Figure 1: Cost and accuracy per epoch of the deep network with augmentation and dropout, 140 training images per landmark.

Model	Train / landmark	Best epoch	Train	Val	Test	Gap	Test top-5	Test GAP	Test macro-F1
Baseline (trial)	73	22	55.6	42.5	43.3	13.1	75.8	–	–
Baseline	105	13	53.6	47.8	46.3	5.8	76.3	0.354	0.450
Baseline + augmentation	105	4	28.4	26.9	25.7	1.5	55.8	0.125	0.239
Deep	105	39	94.1	68.7	69.3	25.4	91.5	0.641	0.693
Deep + augmentation	105	40	81.8	63.2	63.4	18.6	89.5	0.572	0.633
Deep + augmentation + dropout	105	50	68.7	57.8	56.1	10.9	85.3	0.482	0.553
Deep	140	41	95.8	72.5	72.9	23.3	92.0	0.694	0.729
Deep + augmentation + dropout	**140**	**44**	**71.8**	**60.3**	**61.9**	**11.5**	**87.2**	**0.553**	**0.613**

Table 4: All models trained, accuracies in %. Gap is training minus validation accuracy. 105 and 140 training images per landmark correspond to the 150 and 200 subsets of Table 2. The last row is the model kept.

4.4 Error analysis

On the validation set (30 images per landmark) the accuracy of the kept model varies from 10% to 93% depending on the landmark. The best ones are the Nieuwe Waterweg and the Grote Kerk of Breda (93%), the Faisal Mosque and Salève (87%); the worst are St. Lawrence, Toronto (10%), Çufut Qale (17%), the Jurassic Coast, Purana Qila and Ogrodzieniec Castle (33%).

True landmark	Predicted	Images
St. Lawrence, Toronto	Corktown, Toronto	15
Noraduz Cemetery	Hayravank monastery	12
Douthat State Park	Sofiyivsky Park	10
Jurassic Coast	Haleakalā N.P.	7
Ogrodzieniec Castle	Eiffel Tower	6
York River State Park	Sofiyivsky Park	6

Table 5: Most frequent confusions on the validation set, out of 30 images per landmark.

The confusions (Table 5) mostly make sense. Half of the St. Lawrence photographs are predicted as Corktown, the neighborhood next to it in Toronto, which is the case anticipated in Section 2.2: two areas of the same city have no building of their own that sets them apart. Noraduz and Hayravank are both medieval Armenian sites near Lake Sevan, and three of the pairs involve parks or natural sites. Not every confusion has such an explanation, as Ogrodzieniec Castle predicted as the Eiffel Tower shows, but the most frequent ones are between landmarks of a similar kind.

4.5 Predictions

`predict.py` takes any photograph and prints the three most probable landmarks with their probability. Three photographs of the test set:

```
test/47378/09c1766ab27ad60a.jpg  (Eiffel Tower)
   73.5%  Eiffel Tower
   12.1%  Lake Como
    3.7%  Corktown, Toronto
test/168098/13ec9898b26b2571.jpg (Golden Gate)
  100.0%  Golden Gate Bridge
    0.0%  Nieuwe Waterweg
    0.0%  Niagara Falls
test/14915/048adaf6ad345e79.jpg  (St. Lawrence)
   99.1%  Grote of Onze-Lieve-Vrouwekerk
    0.3%  Douthat State Park
    0.2%  Lviv
```

The first two are correct. The third is wrong with 99.1% confidence, the kind of mistake that GAP penalizes the most.

5 Conclusions

A convolutional neural network was trained from scratch with Keras on 50 landmarks of GLDv2. The baseline reached 46.3% test accuracy; diagnosing it as highly biased pointed to capacity, and two convolutions per block raised the test accuracy to 69.3% but turned the problem into high variance, with a gap of 25 points. Augmentation and dropout closed most of that gap, and raising the subset from 150 to 200 images per landmark improved both deep models that were compared. The kept model reaches 61.9% test accuracy, 87.2% top-5 and a GAP of 0.553, with an 11.5 point gap between training and validation.

The most informative result was again the bias–variance diagnosis. Read from the curves, it pointed first to capacity and then to regularization, and each change moved the gap in the expected direction. It also showed the limit of this approach: with 140 images per landmark, regularization reduces variance only by adding bias, and the network cannot both fit and generalize.

The next iteration will add transfer learning from a network pretrained on ImageNet (EfficientNet [8]), which should address what regularization cannot: 140 photographs per landmark are not enough to learn general visual features from scratch. It will also enlarge the dataset, with more photographs per landmark and more landmarks.

References

[1] T. Weyand, A. Araujo, B. Cao and J. Sim. *Google Landmarks Dataset v2 – A Large-Scale Benchmark for Instance-Level Recognition and Retrieval*. CVPR, 2020. `https://github.com/cvdfoundation/google-landmark`

[2] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd ed., O'Reilly Media, 2022.

[3] S. Ioffe and C. Szegedy. *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. ICML, 2015.

[4] M. Lin, Q. Chen and S. Yan. *Network In Network*. ICLR, 2014.

[5] N. Srivastava et al. *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. JMLR 15, 2014.

[6] D. Kingma and J. Ba. *Adam: A Method for Stochastic Optimization*. ICLR, 2015.

[7] K. Simonyan and A. Zisserman. *Very Deep Convolutional Networks for Large-Scale Image Recognition*. ICLR, 2015.

[8] M. Tan and Q. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML, 2019.

[9] Kaggle. *Google Landmark Recognition 2021*, evaluation. `https://www.kaggle.com/competitions/landmark-recognition-2021`

[10] Keras documentation. `https://keras.io/api/`